Coupled Energy-Minimization Smoothing and Topological Mesh Adaptation: Design of an Adaptivity Loop for Norma

Alejandro Mota

August 2026, revised September 22, 2026

Abstract

Energy-minimization smoothing (EMS) improves a mesh by minimizing a hyperelastic distortion energy over the nodal positions, with each element measured against an ideal reference element. Smoothing alone, however, terminates at configurations where no continuous motion of the nodes can further reduce the energy, while the mesh remains poor because the obstruction is topological. Numerical experiments show that alternating EMS with targeted topological operations—edge swaps, collapses, and splits applied to the elements of highest distortion energy—improves mesh quality far beyond what either mechanism achieves in isolation. This note designs such a coupled loop for Norma, informed by a detailed reading of the adaptation logic of the Omega_h library. Three questions are addressed with concrete algorithms: when to trigger topological operations, how to guarantee a monotonic decrease of the total distortion energy across both smoothing and topological phases, and how to detect stagnation of the smoothing phase with a scale-invariant criterion. The central design decision is to use a single distortion functional, well defined for any mesh topology, as the objective of both phases: with a prescribed size or metric field, the metric-conditioned smoothing energy now in Norma is such a functional; without one, an intrinsic equal-volume functional takes its place. The revision of September 2026 fixes the scope (a preprocessing step on four-node tetrahedral meshes, with no transfer of solution state), records the alternatives of wrapping Omega_h or keeping the hand-off to the Sierra tool `improve_mesh`, and states why the topological operations should be accepted on the energy with geometric quality as a floor. The note closes with a step-by-step implementation plan, recommendations for topology and adjacency data structures, and a ranked list of risks. The loop was implemented in September 2026 (Section 11); on a distorted cube it raises the minimum scaled Jacobian from 0.004 to 0.35 and the median from 0.56 to 0.69, where smoothing alone stalls at 0.15 and 0.47. The measurements then qualified the acceptance on the energy: the size operations toward a prescribed target are accepted on the edge length in the target, and the swaps on the cavity minimum of the scaled Jacobian, measured in the metric space under a metric, with the boundary edges of flat patches swapped as well; with these the loop matches the staggered workflow of Norma smoothing and `improve_mesh` on the distorted cube and comes within 0.01 of its mean on a plate refined in four stages.

Contents

1 Motivation and observed workflow **2**

2 Existing infrastructure in Norma **3**

2.1 EMS as a solid-mechanics solve 3

2.2 Assets already in place 4

2.3 Gaps 4

3 The adaptation logic of Omega_h **4**
3.1 Two tiers of thresholds 5
3.2 Control flow 5
3.3 Termination 5

4 The Sierra hand-off and the alternatives to a native loop **5**

5 A single distortion functional **6**
5.1 The obstruction 6
5.2 With a prescribed size or metric field 7
5.3 Without a prescribed target: the equal-volume functional (legacy) 8
5.4 Re-baselining 9

6 Question 1: adaptivity triggers **9**

7 Question 2: energy monotonicity **10**
7.1 The acceptance test 10
7.2 The size operations under the length criterion 11
7.3 The smoothing phase 11

8 Question 3: stagnation detection **13**

9 Architectural plan **14**

10 Data structures and design patterns **15**

11 Implementation and measurements **16**
11.1 What was built 16
11.2 The distorted cube 17
11.3 Comparison with the staggered workflow 18
11.4 The tube 22
11.5 The plate refined toward a sinusoid 22

12 Risks and open questions **23**

13 Summary **24**

1 Motivation and observed workflow

Mesh smoothing by energy minimization treats the mesh as a hyperelastic body whose per-element reference configuration is an ideal element, and minimizes the resulting stored energy over the nodal positions. The energy acts as a barrier against element inversion and drives every element toward the ideal shape. The mechanism is purely geometric: the connectivity is fixed, and only the nodal positions move.

This fixity is also the limitation. Many poor meshes contain configurations—slivers wedged between well-shaped neighbors, vertices of excessive valence, edges shared by too many or too few elements—whose distortion cannot be removed by any continuous motion of the nodes. Smoothing then stalls: the energy plateaus, or the line search fails against the inversion barrier while the gradient remains large. The obstruction is topological, and the remedy is a local change of connectivity.

Conversely, topological operations applied to an unsmoothed mesh are evaluated at poor nodal positions and are frequently rejected or mis-ranked. The two mechanisms are complementary, and the literature on tetrahedral mesh improvement confirms that their combination is substantially stronger than either alone [3, 5].

The workflow this note develops is therefore iterative:

1. run EMS until the energy decrease stagnates;
2. identify the elements of highest distortion energy;
3. apply topological operations to the neighborhoods of those elements, accepting only operations that decrease the energy;
4. resume EMS, which the changed topology now permits to relax further;
5. repeat until neither mechanism makes progress.

Scope. The loop is a preprocessing step: it produces a mesh before any analysis is run on it, so no solution state exists on the mesh and no state transfer is involved. It targets four-node tetrahedra only, which is the element type the smoothing supports and the type the topological operations are written for; a mesh of quadratic tetrahedra is obtained afterwards by conversion, placing the midside nodes at the edge midpoints and projecting those on the boundary to the surface with the level-set machinery already in place. The workflow it replaces is staggered: Norma smooths, the smoothed mesh is passed to the Sierra tool `improve_mesh`, which changes the topology against a geometric quality measure, and the result may be smoothed again (Section 4).

The design questions are precisely where the coupling is delicate: the trigger for step 3, the acceptance criterion that keeps the total energy monotone across steps 1–4, and the stagnation test in step 1. Sections 6–8 address them in turn, after three preparatory sections: an inventory of the machinery Norma already possesses (Section 2), a summary of the decision logic of Omega_h as implemented in its source code (Section 3), and an assessment of the alternatives to a native implementation (Section 4).

2 Existing infrastructure in Norma

2.1 EMS as a solid-mechanics solve

Norma implements EMS not as a separate module but as a mode of the ordinary quasi-static `SolidMechanics` solve. A single substitution in the element loop (`src/model.jl`) replaces each element's reference configuration by an ideal regular tetrahedron before computing the deformation gradient; everything else—assembly, solvers, boundary conditions, output—is the standard machinery. The energy is the Seth–Hill family with exponents $m = n = 2$, which diverges both as the element Jacobian tends to zero and as it tends to infinity, and therefore acts as an inversion barrier. The nodal positions are the unknowns; the minimization runs under the quasi-static integrator as pseudo-time continuation, with matrix-free steepest-descent or L-BFGS steps.

The ideal reference element is built by `create_smooth_reference` (restricted at present to four-node tetrahedra) as a regular tetrahedron of edge length h, where h follows one of four rules: equal volume to the original element, average edge length of the original element, the maximum of these two, or a user-supplied size field $s(t, x, y, z)$. A fifth rule, `metric field` (September 2026), replaces the scalar size by a metric tensor with three principal sizes and a rotation; the ideal element is then anisotropic and the energy is evaluated on the deformation gradient measured in the metric,

so that the orientation of the elements enters the objective (`docs/notes/ems-anisotropic`). In all rules the input geometry is the *original* mesh as read from disk; the consequences of this choice for adaptivity are the subject of Section 5.

Boundary nodes move but remain on the boundary, through either axis-aligned Dirichlet conditions or the `Surface` boundary condition, which represents a boundary surface as an analytic level set $g(\boldsymbol{x}) = 0$ and enforces it by projection: nodes are returned to the surface before assembly (`return_to_surface!`) and the gradient is projected onto the tangent space afterward. This machinery is directly reusable for placing and constraining nodes during topological operations on curved boundaries.

2.2 Assets already in place

Three existing components carry much of the weight of the proposed loop.

1. *A per-element distortion field.* The integrated element energy is stored per block and element in `model.stored_energy`, updated on every evaluation of the model and already written to the Exodus output as an element variable. Step 2 of the workflow—finding the most distorted elements—requires no new computation.
2. *Cross-mesh state transfer.* The module `src/field_transfer.jl` performs a consistent L^2 transfer of nodal fields between meshes of the same geometry, and composes it with the variational recovery of [7] to remap integration-point internal variables. The preprocessing loop of this note does not need it; it is what a future in-simulation adaptivity capability would require, and it connects to the constrained projection of the companion note on internal-variable recovery in admissible spaces (`docs/notes/internal-variable-projection`).
3. *Mid-run mesh replacement.* The module `src/swap.jl` already replaces the mesh of a running simulation: it rebuilds the model, integrator, and solver, and transfers the state to the new mesh. This is the orchestration pattern a topology change must follow, for a structural reason: `initialize_storage` aliases the integrator and solver arrays into the model matrices through raw pointers (`unsafe_wrap`), so any reallocation of the model arrays invalidates the integrator and solver. A topology change must therefore proceed by full rebuild, never by in-place resizing.

2.3 Gaps

Equally important is what does not exist. The mesh is never held in memory as a mutable object: the `ExodusDatabase` handle is the source of truth, connectivity is re-read from the file on every access, and no function writes coordinates or connectivity back. There are no adjacency structures of any kind—no node-to-element, element-to-element, or edge data—so there is at present no way to enumerate the cavity around an edge. There is no element quality module; the de facto quality measure of the code base is the smoothing energy itself. (Point location, a linear scan when this note was first written, is now a uniform-grid search in `src/spatial_search.jl`.) These gaps define Step 3 of the implementation plan in Section 9.

3 The adaptation logic of Omega_h

Omega_h [1, 2] is a metric-driven simplex-adaptation library whose decision logic is compact and, as verified directly against its source code, organized around three ideas: a two-tier system of

thresholds, cavity operators filtered by acceptance criteria, and progress-based termination. This section records the logic precisely, because Section 6 transplants it.

3.1 Two tiers of thresholds

The options structure `AdaptOpts` carries *desired* and *allowed* values with distinct roles: desired values select candidates for work, allowed values are hard limits that no accepted operation may violate. The defaults are: metric edge lengths desired within $[1/\sqrt{2}, \sqrt{2}]$, length allowed up to $2\sqrt{2}$; element quality desired at least 0.30 (three dimensions), quality allowed at least 0.20. There is no lower allowed length: short edges are only ever "desired" away. The quality measure is the metric-conditioned mean ratio, an intrinsic shape measure equal to one for a metric-equilateral simplex and non-positive for a degenerate or inverted one.

3.2 Control flow

The top-level `adapt()` first evaluates an "already satisfactory" predicate—minimum quality at least the desired quality *and* all metric edge lengths within the desired band—and returns without touching the mesh if it holds. Otherwise it runs two phases.

1. *Length phase.* Alternate refinement of edges longer than $\sqrt{2}$ and collapse of edges shorter than $1/\sqrt{2}$, repeating until neither operator changes anything. Acceptance in this phase enforces only the hard limits: a refinement is accepted if the worst new element stays above the allowed quality; a collapse additionally may not create edges beyond the allowed length. Quality may therefore *decrease* in this phase, down to the floor.
2. *Quality phase.* While the minimum quality is below the desired value: attempt edge swaps; only if a full swap pass changes nothing, attempt sliver collapses; if both fail in the same pass, report failure and stop. Acceptance in this phase is strict improvement: a swap or sliver collapse is kept only if the minimum quality of the new cavity *strictly exceeds* that of the old cavity. This strict filter is the only monotonicity guarantee in the library.

Candidates for the quality phase are the elements below desired quality *dilated by four layers of adjacent elements*, because the operation that repairs a bad element frequently lies on a neighboring entity. Boundary edges are never swapped. Conflicts between overlapping cavities are resolved by a Luby-style independent set in which the cavity of higher quality wins, with a deterministic global-identifier tiebreak; candidates not selected simply wait for the next pass.

3.3 Termination

There is no iteration counter anywhere in `adapt()`. Every operator returns false exactly when no candidate survives its filter chain, and each phase loop exits when its operators return false. Termination is thus purely progress-based: the loop runs precisely as long as some operation can be accepted. When the quality goal is unreachable, the library says so and returns the mesh in its best achieved state rather than looping.

4 The Sierra hand-off and the alternatives to a native loop

Three routes exist: keep the hand-off to `improve_mesh`, wrap Omega_h into Norma, or implement the operators natively. The facts about the two external tools, read from their sources in September 2026, are these.

`improve_mesh` and `emend`. `improve_mesh` is a thin driver (about 2,500 lines: an input parser, STK mesh creation, and optional CAD geometry through `m2g`) over the Sierra library `emend` (93 files, about 10,000 lines). `emend` implements edge swaps, face swaps, edge collapses (with a variant for mandatory node removal), edge refinement, independent-set selection of non-conflicting cavities, control of the deviation from the geometry (`edgeGeomDeviationTol`, `sharpFeatureAngleTolInDegrees`), and midnode straightening for quadratic elements. Its parameters follow the Omega_h logic: desired and allowed relative edge lengths at $1/\sqrt{2}$ and $\sqrt{2}$, a quality below which improvement is attempted, iteration caps per operator, and termination when the quality stops improving. It accepts nodal isotropic and anisotropic size fields, Cartesian-aligned and cylindrical size fields, and a displacement field. Its quality is an abstract `QualityMetric` compared through `is_first_quality_metric_better_than_second`, so the measure is pluggable, but acceptance is always the comparison of the worst element of a cavity before and after the operation. It contains no field transfer and no code from Omega_h: it is an independent implementation of the same decision logic on STK, and it is not open source.

Omega_h. C++14 with CMake; optional MPI, OpenMP, or CUDA; tetrahedra and triangles; anisotropic metric fields with gradation control; refinement, coarsening, swapping, and sliver coarsening under `AdaptOpts`; Exodus input and output through SEACAS; boundary geometry through EGADS or through entity classification alone. The quality is the metric-conditioned mean ratio, fixed in the library; the one extension hook, `UserTransfer`, concerns fields, not the acceptance criterion. There is no C interface.

Comparison. Table 1 sets the two routes that would bring the operations into Norma against each other.

Assessment. Keeping the hand-off is mature and in use, but it cannot be part of a public Norma workflow, and the two tools will always optimize different quantities (the energy in Norma, a geometric quality in `emend`), which is the source of the inefficiency of the staggered loop. Wrapping Omega_h buys a mature adapter quickly but reproduces that staggered loop with a different external tool and adds a build dependency. The value of doing this inside Norma is one objective for both phases, and that requires the native route. The native route is a bounded piece of work because its hard parts exist or are specified: the energy kernel and the metric-space ideal element are in the code, the geometry constraints exist, the orchestration pattern exists, and the operators for four-node tetrahedra are well documented (an edge loop of at most seven elements admits a small table of triangulations; collapses and splits are local). `emend` is ten thousand lines because it also handles quadratic elements, CAD geometry, STK, and parallel independent sets, none of which are in the first version here. The recommendation is the native route, with Omega_h in one narrow role: its command-line tools read and write Exodus, so a mesh and a metric can be run through it and its length and quality histograms compared with ours at no integration cost, and with `improve_mesh` as the reference for the validation cases of Section 9.

5 A single distortion functional

5.1 The obstruction

The design problem that governs the entire coupling is that the smoothing objective under the `equal volume`, `average edge length`, and `max` rules is tied to the original mesh: the ideal reference of

	Wrap Omega_h	Native operators in Julia
Work to first result	A C shim (mesh arrays in and out, metric per vertex, options, the `adapt` call) of a few hundred lines plus a `ccall` layer, packaged as an optional extension; about two weeks	Steps 3 to 7 of Section 9; three to four thousand lines of Julia with tests; two to three months to a validated loop
Build and distribution	A C++ dependency with CMake in a code that runs wherever Julia runs; CI on Ubuntu and macOS must build it; a binary recipe for users	None
Acceptance criterion	Fixed: metric-conditioned mean ratio and the length band; an energy criterion requires patching the library	Any: the energy criterion of Section 7, a geometric floor, or both
Coupling with smoothing	Loose: the two tools optimize different objectives	Tight: one objective, alternating phases, monotone descent
Boundary geometry	Classification only, or EGADS; the analytic `Surface` level sets would have to be re-expressed	`Surface` level sets and node and side sets reused directly
Parallelism	MPI and threads	Serial at first; threads later through independent sets
Maturity	High, deterministic, with stated quality guarantees	To be earned: cavity validity, orientation, and boundary preservation need careful tests
Public repository	Yes (BSD license)	Yes

Table 1: Bringing topological operations into Norma: wrapping Omega_h against a native implementation.

each element is built from the geometry of that element in the mesh as read from disk. After a topological operation, a new element has no original counterpart, so the objective is undefined on it; and even where defined, an energy computed against history-dependent targets cannot be compared across a topology change, because the comparison would be between different functionals. A monotonicity guarantee requires a single functional, defined for every admissible pair of topology and nodal positions, that both phases of the loop demonstrably decrease. There are two such functionals, depending on whether a target is prescribed.

5.2 With a prescribed size or metric field

When the target is a size field $h(\boldsymbol{X})$ or a metric field $\boldsymbol{M}(\boldsymbol{X})$ (`docs/notes/ems-anisotropic`), the ideal element of any tetrahedron is the metric sampled at its centroid, for a new element exactly as for an old one. The smoothing energy

$$D(\mathcal{T}, \boldsymbol{x}) \;=\; \sum_{e \in \mathcal{T}} V^e_{\boldsymbol{X}}\, \psi(\boldsymbol{F}_{U,e})\,, \qquad \boldsymbol{F}_{U,e} = \boldsymbol{F}_M \boldsymbol{F}_e \boldsymbol{F}_M^{-1}, \tag{1}$$

with $V^e_{\boldsymbol{X}} = h_1 h_2 h_3 V_{\text{unit}}$ the volume of the ideal element and $\boldsymbol{F}_{U,e}$ the deformation gradient measured in the metric, is therefore defined for every admissible pair $(\mathcal{T}, \boldsymbol{x})$ without reference to the history of the mesh. It measures size, shape, and orientation against the target in one number: it vanishes

only for a unit element of the metric, it diverges at inversion, and it reduces to the isotropic energy for equal sizes. This is the functional the loop should use whenever a target is prescribed, and it changes two things in what follows. First, an operation that moves the mesh toward the target size is, in the sum over the mesh after smoothing, a decrease of the same functional the smoother minimizes: a too-large element has a large volumetric energy and its children, each nearer the target volume, have less. The cavity test sees only part of this. A split bisects the elements around the edge and roughly halves their scaled Jacobian, and the shape penalty of the halves, before the smoothing and the swaps that repair them, outweighs the size gain until the edge is far longer than the target: measured on the plate of Section 11.5, with equal bulk and shear moduli, every split of the edges beyond the band raised the cavity energy, and the loop stalled at about 1.5 times the target. The size operations are therefore accepted on the edge length in the target (Section 7.2) and the energy decides the shape within the band. Second, the volume floor of the rules `size field` and `metric field`, which scales the target up to the volume of the original element, exists only because smoothing cannot change the number of elements; in the coupled loop it is unnecessary and the `unrestricted` variants are the right ones. The target is held fixed during a smoothing phase, sampled at the centroids of the elements at the start of the phase, and re-sampled at every phase boundary, as the anisotropic note describes; a new element created by a topological operation receives the metric at its centroid when it is created. For thresholds the natural density is $\hat{w}_e = \psi(\boldsymbol{F}_{U,e})$, the energy per unit ideal volume, which is dimensionless with unit moduli.

5.3 Without a prescribed target: the equal-volume functional (legacy)

The rules that take the target from the original element (`equal volume`, `average edge length`, `max`) are legacy rules since September 2026: new inputs prescribe a size or metric field, and the functional of Section 5.2 is the one the loop uses. The equal-volume functional is kept here for meshes smoothed with the legacy rules. Without a target, the mesh should keep its local size and only improve its shape. Let $\mathcal{T}$ denote the mesh topology and $\boldsymbol{x}$ the nodal positions. Define

$$D(\mathcal{T}, \boldsymbol{x}) \;=\; \sum_{e \in \mathcal{T}} W_e(\boldsymbol{x}), \qquad W_e(\boldsymbol{x}) \;=\; \int_{\Omega_e^{\mathrm{ideal}}} \psi(\boldsymbol{F}_e)\, \mathrm{d}V, \tag{2}$$

where ψ is the Seth–Hill energy density already in use, $\boldsymbol{F}_e$ is the deformation gradient of element e measured from its ideal reference, and—this is the decision—the ideal reference of each element is the regular tetrahedron of *equal volume to that element's own current volume*. This is the existing `"equal volume"` rule of `create_smooth_reference`, applied to the current rather than the original geometry.

With this choice the energy of an element depends only on its own present shape: it is invariant under uniform scaling, vanishes exactly for a regular tetrahedron, and diverges at degeneracy. The functional (2) is therefore *intrinsic*—a function of $(\mathcal{T}, \boldsymbol{x})$ alone, with no dependence on mesh history—and is well defined for any tetrahedral mesh whatsoever. It belongs to the same class as the mean-ratio quality of Omega_h, which is likewise a scale-invariant shape measure of the current element; the difference is that (2) is an energy with an inversion barrier rather than a bounded quality, which is precisely what makes it minimizable by the existing solvers. For threshold purposes define the normalized density

$$\hat{w}_e \;=\; \frac{W_e}{V_e}, \tag{3}$$

with V_e the current element volume, so that $\hat{w}_e = 0$ for an ideal element and $\hat{w}_e \to \infty$ at degeneracy. Note the orientation: minimum quality in Omega_h corresponds to *maximum* energy density here.

Remark 1. *Size-targeting is excluded from* (2) *by construction: this functional keeps whatever size the mesh has. Operations toward a target size belong to the functional* (1) *of Section 5.2, where they are decreases of the objective and need no separate phase. The two functionals are not mixed within one loop: a run either has a prescribed target or it does not.*

5.4 Re-baselining

Re-baselining concerns the equal-volume functional; with a prescribed target the ideal element is already fixed by the field. The smoothing solver requires a *fixed* objective during a solve: if the ideal reference tracked the current volume continuously, the target would move with the unknowns. The resolution is re-baselining at phase boundaries: at the start of each smoothing phase, set the smoothing reference configuration to the current coordinates and build the equal-volume ideal references from it. The phase objective then coincides with $D(\mathcal{T}, \cdot)$ at the start of the phase and remains a fixed, minimizable functional throughout it. The relation between the frozen-target phase objective and the intrinsic functional is that of a majorize–minimize surrogate; the safeguard of Proposition 1 converts the practical descent of D into a guaranteed one.

6 Question 1: adaptivity triggers

The translation of the Omega_h logic into the energy setting is summarized in Table 2 and specified below.

Omega_h	Proposed EMS loop
mean-ratio quality (max. 1, degenerate ≤ 0)	energy density $\hat{w}_e$ (min. 0, degenerate $\to \infty$)
`min_quality_desired` (candidate selection)	$\hat{w}_{\text{des}}$: element is worth working on
`min_quality_allowed` (hard floor)	$\hat{w}_{\text{all}} > \hat{w}_{\text{des}}$: hard ceiling
metric length band $[1/\sqrt{2}, \sqrt{2}]$	same band selects split and collapse candidates
`nsliver_layers` $= 4$ adjacency layers	same: dilate candidate set by n_{layers} rings
swap first, sliver-collapse as fallback	same order
strict cavity-quality improvement filter	strict cavity-energy decrease (Section 7)
progress-based termination, no counters	same

Table 2: Correspondence between the Omega_h adaptation logic and its proposed energy-based analogue. The orientation inverts: minimum quality corresponds to maximum energy density.

1. *Trigger for the topology phase.* Enter the topology phase when the smoothing phase has stagnated in the sense of Section 8 *and* $\max_e \hat{w}_e > \hat{w}_{\text{des}}$. This is the "already satisfactory" predicate of Omega_h with quality replaced by energy density; with a prescribed target the energy density already contains the length clause, since an element of the wrong size has a large volumetric energy.
2. *Candidate set.* All elements with $\hat{w}_e > \hat{w}_{\text{des}}$, dilated by n_{layers} rings of element adjacency (default four, following Omega_h); the operation candidates are the interior edges of this set. Boundary edges are excluded from swapping; they participate in collapses and splits only under the surface constraints of Section 9.
3. *Order within a pass.* Swaps first; collapses only when a full swap pass accepts nothing. Swaps move no nodes and therefore compose trivially with smoothing; collapses remove resolution

and are the fallback. Splits participate where a bisection lowers the cavity energy (Remark 2); with a prescribed target, edges longer than $\sqrt{2}$ in the metric are the split candidates and edges shorter than $1/\sqrt{2}$ the collapse candidates, in addition to the edges of the high-energy cavities.

4. *Outer termination.* The coupled loop ends when the smoothing phase reports stagnation or convergence and the subsequent topology phase accepts zero operations, or when smoothing converges with $\max_e \hat{w}_e \le \hat{w}_{\mathrm{des}}$. Termination is progress-based, as in Omega_h; a large safety cap on outer iterations is retained for defense but is not the semantic terminator. When the goal is unreachable, the loop reports it and returns the best achieved mesh.

7 Question 2: energy monotonicity

The reading of the Omega_h source localizes its monotonicity exactly: strict improvement is guaranteed only where the strict filter is applied—swaps and sliver collapses—while refinement and size-driven collapse may degrade quality to the floor. Adapting the criterion to the EMS framework therefore means applying a strict-decrease filter, stated in energy, to *every* operation of the quality phase, and pairing it with a monotone smoothing phase. Both are stated against the intrinsic functional (2).

7.1 The acceptance test

A topological operation replaces the elements C_{old} of a cavity by new elements C_{new} at fixed nodal positions $\boldsymbol{x}$ (for a split, the new node is placed and locally relaxed first; see Remark 3). The operation is accepted if and only if

$$\sum_{e \in C_{\mathrm{new}}} W_e(\boldsymbol{x}) \;\le\; \sum_{e \in C_{\mathrm{old}}} W_e(\boldsymbol{x}) \;-\; \delta, \qquad \text{and} \qquad \max_{e \in C_{\mathrm{new}}} \hat{w}_e \;\le\; \hat{w}_{\mathrm{all}}, \tag{4}$$

with $\delta > 0$ a small relative margin (for example 10^{-8} times the old cavity energy) that precludes cycles of operations accepted at zero gain. The cavity *sum* replaces the cavity minimum of Omega_h deliberately: the sum is the restriction of the global functional to the cavity, so local acceptance implies global descent, which is the property the loop needs. The ceiling $\hat{w}_{\mathrm{all}}$ plays the role of the allowed-quality floor; element inversion is excluded automatically because the Seth–Hill barrier sends $\hat{w}_e$ to infinity there.

Energy and geometric quality. The energy is the right quantity to accept an operation on, for three reasons. It is one functional for both phases, so the alternation cannot cycle and terminates (Proposition 1); a geometric criterion for the topology and an energy for the smoothing are two objectives, each phase can undo the other's progress, and termination then rests on iteration caps, which is what `emend` does with its `maxOuterIterations`. It measures size, shape, and orientation at once: the scaled Jacobian and the mean ratio measure shape only, need a separate length band for size, and are blind to the orientation of an element relative to an anisotropic target unless the element is first mapped into the metric space. And it carries the inversion barrier. The geometric measures keep two roles. The first is the hard floor: the test compares cavity sums, and a proposal could lower the sum while creating one element worse than the analysis code tolerates, so the second condition of (4) may be stated, in place of or in addition to $\hat{w}_{\mathrm{all}}$, as the minimum scaled Jacobian or mean ratio the solver requires; both are cheap, and the geometric one is what the analysts already ask for, so reporting it keeps the results comparable with those of `improve_mesh`. The second is candidate selection through the length band, as stated above. One property of the energy to keep

in view: it is not scale-free across elements of different target sizes, because the element energy is the density times the ideal volume. Thresholds and the floor are therefore on the density $\hat{w}_e$, while the cavity comparison is on the sums, which weight elements by their target volume as a global objective should.

7.2 The size operations under the length criterion

The argument above, that one functional decides every operation, holds for the swaps and for the shape-driven collapses and splits, and it fails for the size operations, for a reason the measurements of Section 11.5 make plain. Consider a smoothed mesh whose edges are r times the target. Bisecting an element halves its size mismatch but also roughly halves its scaled Jacobian, since the half has the height of the parent on half its base, and the Seth–Hill energy with equal bulk and shear moduli charges more for that shape than it returns for the size until r is about 1.5 to 2 (the volumetric term grows as the fourth power of the mismatch, the isochoric term is already of order one for the half). The smoothing and the swaps that follow repair the halves, but the cavity test cannot foresee them. A heavier bulk modulus lets the splits through (with the bulk modulus 64 times the shear modulus, 897 of 906 long-edge splits lowered the cavity energy on the plate) at the price of the smoothing, which then trades shape for size: over three refinement stages the median scaled Jacobian fell from 0.50 to 0.33 and the minimum to 0.025. The two roles of the functional, to accept a refinement and to shape the elements, want different weights, and no single pair serves both.

Under `size criterion: length` the loop therefore does what every metric-based adaptation code does: an edge longer than $\sqrt{2}$ in the target is split and an edge shorter than $1/\sqrt{2}$ is collapsed on the length alone, subject to the geometric floor, the set and surface constraints, and a finite energy (the validity check), while the swaps and the shape-driven collapses and splits keep the energy test (4). Two guards make the length criterion consistent. Without them the loop churned at hundreds of operations per pass: a split of an edge in a flat cavity creates edges from the new node to the ring shorter than the band, which the next pass collapses, and a collapse of a short edge onto a far node creates edges longer than the band, which the next pass splits; and in a pass with no accepted swap the shape-driven collapses, judged by the energy that favors fewer and larger elements, removed what the length-driven splits had added. So under the length criterion no collapse or split, length-driven or shape-driven, may create an edge outside the band: the target decides the sizes and the energy decides the shape within the band. With the guards the passes of a topology phase end at zero operations and the adaptivity iterations converge (on the first stage of the plate: 1126, 391, 142, 68, 44, 38, 23, 17 operations).

The floor interacts with refinement. Because a bisection halves the scaled Jacobian, a floor of 0.35 on a mesh whose median is 0.58 rejects nearly every split (on the plate, 981 of 29797 splits passed it, against 29222 without a floor), so a refinement stage wants a low floor, 0.15 in the examples, and relies on the smoothing and the swaps to raise the quality afterwards. The floor of 0.35 that served the distorted cube, where the target equals the mesh size and the size operations are few, is a quality guard for that regime.

7.3 The smoothing phase

The L-BFGS path of the smoothing solver already performs an Armijo backtracking line search on the energy, so each smoothing phase is monotone in its own frozen-target objective; the steepest-descent path currently backtracks on the residual norm instead and should use the energy merit for this loop. With re-baselining at phase start (Section 5), the phase objective agrees with $D(\mathcal{T}, \cdot)$

at the initial iterate. Because the phase objective holds its targets fixed while D is intrinsic, the phase is completed by a safeguard: evaluate D once at phase end, and roll the phase back if D increased. The safeguard costs one evaluation and is expected to be inert in practice; its purpose is to turn practical descent into provable descent.

Proposition 1 (Global descent and finite termination of the topological work)**.** *Let the coupled loop consist of smoothing phases with the end-of-phase safeguard and topological operations accepted by the test* (4)*. Then the sequence of values of D sampled at phase boundaries and after each accepted operation is nonincreasing and bounded below by zero, hence convergent; each accepted operation decreases D by at least δ times the corresponding cavity energy at acceptance, so if δ is enforced against a fixed positive reference energy the number of accepted topological operations is finite.*

Proof. Only cavity elements change in a topological operation, so (4) implies D decreases by at least δ across it. The safeguard makes each smoothing phase nonincreasing in D by construction. A nonincreasing sequence bounded below converges, and a uniform positive decrement per accepted operation bounds their number. □

The guarantee lives in exactly one place: a single acceptance routine implementing (4), called by every operator. No operator carries its own criterion.

Remark 2 (Splits participate honestly)**.** *Because the equal-volume energy is scale-invariant, bisecting a badly stretched element into two better-shaped children genuinely lowers the cavity energy, and refinement needs no exemption from the test* (4)*. With a prescribed target the same holds for size-driven refinement and coarsening, since the metric-conditioned energy* (1) *penalizes the wrong size directly.*

Remark 3 (Local relaxation before the test)**.** *For a split, place the new node at the edge midpoint—projected to the surface by* `return_to_surface!` *when the edge lies on the boundary—and run a few iterations of cavity-local smoothing with the cavity boundary fixed before evaluating* (4)*. The cost is a handful of element evaluations; the benefit is that each operation is judged at its locally relaxed optimum rather than at an arbitrary initial placement, which raises the acceptance rate substantially. This is the natural advantage of an energy framework over the position-free acceptance tests of Omega_h, and it is the precise mechanism by which the alternation outperforms either method alone: each accepted operation unlocks relaxation that the previous topology forbade.*

The same relaxation applies to a swap or a collapse that (4) *refuses at the fixed positions (*`cavity relaxation: true`*, October 30, 2026). The interior nodes of the cavity—those in no node set and on no side set—are relaxed one at a time against the new connectivity, each by up to five descent steps on the energy of the elements around it, with the gradient of the energy with respect to the node—the element internal force of the smoothing material, $\sum_q \boldsymbol{P}\,\nabla N_a\,dV$, from one evaluation of the elements around the node—and the step halved until the energy decreases, as for the node of a split; when after the first step over the nodes the energy of the new and surrounding elements is still above* 1.5 *times that of the old and surrounding ones the relaxation stops and the operation is refused (on the awful cube 98% of the operations the full relaxation accepts are below that ratio after the first step, and 71% of the relaxations tried are above it). The test then compares the old elements with the new ones at the relaxed positions, both together with the elements around the moved nodes that the operation keeps, since those change with the positions; so an accepted operation still lowers the global functional, and Proposition 1 holds with the moved nodes included in the step. A swap accepted at the fixed positions is not relaxed, since the smoothing phase moves the nodes anyway. Measured on the smoothed cube and awful cube of the Delaunay study (*`~/Projects/Norma-EMS-Topology-2026-10-30/delaunay/relaxation.md`*),*

with the swap phase alone at all candidates: at the fixed positions the phase accepts 51 swaps on the cube and lowers the energy by 2%; with the relaxation it accepts 734, moves 907 of the 1800 nodes, and after the smoothing that follows the energy is 5.69×10^2 *against* 7.24×10^2*, the minimum scaled Jacobian* 0.52 *against* 0.45*, and the mean* 0.786 *against* 0.762*. On the awful cube, whose smoothing needs 500 steps to converge, the energy after smoothing is* 9.92×10^3 *against* 1.51×10^4*, the minimum scaled Jacobian* 0.40 *against* 0.36*, and the mean* 0.70 *against* 0.65*. Started from the Delaunay connectivity of the same positions instead, the relaxed phase ends within 2% in energy of these values. The cost is the obstacle: every refused candidate is relaxed, and a refusal is forgotten as soon as a node around it moves. On the loop of* `awful-cube-adaptive.yaml` *under the energy criteria the topology phases took 117 s with the gradient by central differences against 2.4 s without the relaxation; the analytic gradient brought them to 51 s with the same mesh to the last digit, and the stop after the first step to 26 s, with the final energy 7.08 against 9.51, the minimum scaled Jacobian 0.38 against 0.36 and the mean 0.72 against 0.68 at the same element count. With the boundary nodes relaxed as well, sliding on the surfaces of their Surface conditions and keeping the components their Dirichlet conditions fix, the loop reaches a minimum of* 0.42 *and a mean of* 0.73 *with 5406 elements, against* 0.41 *and* 0.70 *for the staggered workflow with* `improve_mesh` *and* 0.40 *and* 0.70 *for the shipped configuration of the example at 8258 elements, in 40 s of topology time. A bound on the energy ratio of the new to the old cavity at the fixed positions does not select the candidates: the accepted operations have a median ratio of 2.6 and a tail to* 10^{13} *(new elements nearly degenerate at the fixed positions that the moved nodes open up). The relaxation is off by default at a factor of 11 in the time of the topology phases; the number of relaxations tried, 120 678 for 5642 accepted, is the remaining lever.*

8 Question 3: stagnation detection

The smoothing solver currently stops on a gradient-norm test with absolute and relative tolerances—a *convergence* criterion. Stagnation, the absence of significant further decrease, requires a criterion that is scale-free, tolerant of plateaus, and inexpensive. The smoothing phase should end at the first of the following three events, and report which one occurred.

1. *Windowed relative decrease.* Let W_n denote the phase objective at iteration n and let m be a window length (default $m = 10$). Stop when

$$W_{n-m} - W_n \;\le\; \tau_{\text{stag}} \big(W_0 - W_n\big), \qquad \tau_{\text{stag}} = 10^{-3} \text{ by default.} \tag{5}$$

   In words: the decrease achieved over the last m iterations is a negligible fraction of the decrease achieved since the phase began. The test is invariant under affine rescaling of the energy and under problem size, and the window absorbs the short plateaus that quasi-Newton iterations produce near barrier-dominated configurations. Since Armijo backtracking makes $\{W_n\}$ monotone, the left side is nonnegative and the test is well posed.

2. *Convergence proper.* The existing projected-gradient test passes: the phase ended because the mesh is smooth, not because it is obstructed.

3. *Topological lock.* The line search backtracks to failure, or the non-positive-Jacobian guard trips repeatedly, while the projected gradient remains large. This is not convergence: it is the inversion barrier certifying that no continuous nodal motion can improve the mesh from the current configuration. It is the strongest possible trigger for the topology phase and should be detected and reported explicitly.

The soundness of (5) rests on the standard descent inequality: for a continuously differentiable objective with Armijo backtracking, the per-iteration decrease is bounded below by a multiple of the squared projected-gradient norm times the accepted step [6]. Sustained failure to decrease therefore certifies either a vanishing projected gradient (event 2 in disguise) or a collapsing step length against the barrier (event 3); the windowed test detects both with a single properly scaled quantity and no problem-dependent tuning.

Distinguishing the three events gives the outer loop its terminator at no additional cost: event 2 with $\max_e \hat{w}_e \leq \hat{w}_{\mathrm{des}}$ means the loop is finished; events 1 and 3 call for a topology phase; a topology phase that accepts nothing after event 1 or 3 means the loop has reached the best this machinery can achieve, which is reported rather than concealed—the exact analogue of the explicit failure report of Omega_h when its quality goal is unreachable.

9 Architectural plan

The implementation follows the repository convention of paired files: `src/topology_types.jl` and `src/topology.jl` for the mesh and its adjacencies, `src/adapt_types.jl` and `src/adapt.jl` for the operators and the loop. The steps are ordered so that each is independently testable, with the first coupled loop appearing at Step 4. Steps 1 to 7 were carried out between September 18 and 19, 2026 (Section 11).

1. *Energy kernel on subsets.* Factor the per-element energy evaluation out of the assembly loop into a kernel callable on an arbitrary subset of elements at given nodal positions, with the metric-sampled ideal element of Section 5.2 for any element and the equal-volume re-baselined variant as the fallback, and register the density $\hat{w}_e$ as an Exodus element variable. Testable immediately against `model.stored_energy` on the existing `examples/ems` cases.

2. *Stagnation criterion.* Add the windowed test (5) and the lock detection to the solver stopping logic, returning a phase outcome (converged, stagnated, locked) rather than a Boolean, and switch the steepest-descent merit to the energy. Independently testable: the strongly distorted examples should report the lock outcome where they currently exhaust iterations.

3. *In-memory topology and adjacency.* A topology cache built once per topology phase from the Exodus-read connectivity: coordinates, per-block connectivity, and the three adjacencies the code base lacks—element-to-element across faces (by sorted-face matching), node-to-elements in compressed sparse row form, and an edge table mapping each edge to its incident elements. Boundary faces are identified as unpaired faces and cross-checked against the side sets; node-set and side-set membership is stored as per-entity flags so operations can update it. Four-node tetrahedra only, matching the existing restriction of `create_smooth_reference`.

4. *Edge swap and the acceptance test; first coupled loop.* Implement the three-dimensional edge swap: extract the loop of elements around an edge (at most seven, following Omega_h), enumerate the tabulated triangulations of the surrounding polygon, and select the best; implement the single acceptance routine (4); wire the outer driver (smoothing phase, candidate collection, swap passes, repeat). Swaps move no nodes and need no state transfer, so this milestone alone should reproduce most of the observed benefit of the alternation.

5. *Edge collapse.* Sliver removal under the acceptance test (4); collapses on the boundary are restricted to boundary edges with the surviving node kept on the surface through the `Surface` level-set machinery; nodes belonging to node sets are never removed; an orientation guard

rejects collapses that would invert any element (already implied by the ceiling, but checked cheaply before the energy evaluation).

6. *Edge split and the size target.* Midpoint placement with surface projection for boundary edges, cavity-local relaxation per Remark 3, then the acceptance test. With a prescribed target, edges longer than $\sqrt{2}$ or shorter than $1/\sqrt{2}$ in the metric join the candidate set, the target is used `unrestricted`, and a new element takes the metric at its centroid; these operations pass through the same test as the others. The acceptance case is the tube of the anisotropic note with a radial size of a quarter of the wall, which smoothing alone cannot reach.

7. *Orchestration and output.* After each topology phase, rebuild the model, integrator, and solver through the existing mesh-replacement pattern of `src/swap.jl`, which avoids the pointer-aliasing hazard by construction. Write each adapted mesh as a fresh Exodus file in a numbered sequence: the Exodus format cannot change element counts within a file, so per-phase files are the practical route, and the replacement machinery already validates it. Input schema: an `adaptivity` block carrying $\hat{w}_{\mathrm{des}}$, $\hat{w}_{\mathrm{all}}$, the stagnation window and tolerance, the number of adjacency layers, and per-operator switches, mirroring `AdaptOpts`.

8. *Validation against the hand-off.* For the cube, the tube, and the strongly distorted examples, run the same input mesh and target through `improve_mesh` and through the native loop and compare the energy and scaled-Jacobian histograms of the two outputs; run Omega_h on the same Exodus files where a metric is prescribed and compare its length and quality histograms. The first milestone at which this comparison is meaningful is Step 4, since swaps alone, alternated with the existing smoother, are expected to show most of the benefit.

9. *Out of scope.* In-simulation adaptivity, which would invoke the driver from the evolution loop and transfer the solution state through `field_transfer.jl`, and quadratic elements, which are obtained by conversion after the loop. Both are noted here so that the data structures of Section 10 do not preclude them.

10 Data structures and design patterns

1. *Rebuild per phase, not incremental maintenance.* Omega_h rebuilds its topology arrays wholesale after each operator pass rather than patching them. The same pattern is right here: topology phases are rare relative to smoothing iterations, wholesale rebuild is simple and cache-friendly, and it eliminates the entire class of stale-adjacency defects. Within a pass, operations are applied through tombstones—dead elements and nodes are marked, new ones appended—and the arrays are compacted once at the end of the pass.

2. *One cavity abstraction, one acceptance test.* Every operator produces a cavity proposal: the identifiers of the old elements, the proposed new connectivity, and optionally a new node. A single routine evaluates (4) and applies or discards the proposal. The monotonicity guarantee then has exactly one owner in the code.

3. *Sequential priority order now, independent sets later.* The Luby-style independent set of Omega_h exists for distributed parallelism. In serial, sorting the candidates by cavity energy in decreasing order and applying them sequentially is simpler and equally effective; a cavity that touches an element already modified in the current pass is deferred to the next pass, exactly as unselected Omega_h candidates wait for theirs. If the topology phase is ever

threaded, adopt the independent set with the same comparator: the better cavity wins, with a deterministic identifier tiebreak.

4. *Exodus out of the hot path.* When this note was first written the connectivity was re-read from the database handle inside the assembly loop; the per-block element data of the model now hold it in memory, and the topology cache of Step 3 is the source of truth during adaptation, with Exodus used for input and output at phase boundaries only.

5. *Outcome types over Booleans.* The solver reports converged, stagnated, or locked; each operator pass reports its number of accepted operations; the driver's control flow is written entirely against these outcomes. This makes the progress-based termination of Section 6 auditable in the logs, in the same way that each Omega_h operator returns false exactly when its filter chain leaves no candidate.

11 Implementation and measurements

11.1 What was built

- *Energy kernel on subsets* (`element_energies`): the energy of any set of elements given by connectivity and nodal positions, evaluated as the assembly evaluates it, with the target sampled at the positions the caller supplies and an infinite energy for an inverted or degenerate element; the inversion is judged on the determinant of the deformation gradient itself, since for a nearly flat element it can differ in sign by roundoff from the determinant of the current-configuration Jacobian. The smoothing output carries the density $\hat{w}_e$ as the element variable `energy_density`.

- *Topology* (`MeshTopology`): coordinates, connectivity, node-to-element, edge and face tables, boundary edges, node sets and side sets as flags and faces; tombstones within a pass, compaction with renumbering maps, and Exodus output with the sets and their names.

- *Edge swap*: the triangulation of the ring polygon of least score submitted to the acceptance test. The polygon of n nodes has C_{n-2} triangulations, with C_m the Catalan numbers (3.5×10^7 for $n = 18$), and the ring size is not bounded. The score of a triangulation is the sum of the energies of the tetrahedra of its triangles, or under the scaled Jacobian criterion the maximum over them of the negative scaled Jacobian. Each condition of admissibility concerns one triangle or one chord: the validity conditions below, a chord that is not yet an edge, and new faces that are not yet faces of the mesh. The least score is therefore found by dynamic programming over the sub-polygons [4], with $\binom{n}{3}$ evaluations of a triangle and without enumerating the triangulations (issue #247). Positive orientation of the new tetrahedra is not a sufficient validity test: an ear at a reflex vertex of the ring polygon gives positively oriented tetrahedra that coincide with an element outside the cavity (found as a face shared by four elements). A triangulation is valid when every triangle has the orientation of the ring polygon in the projection along the edge and the edge nodes lie on opposite sides of it.

- *Edge collapse*: one node of an edge removed onto the other, which keeps its position; the removed node must carry no node set the survivor lacks, and a boundary node may move only along a boundary edge onto a node of the same surfaces, so surfaces and feature lines are preserved and corners never removed; the side sets are updated.

- *Edge split*: a node at the midpoint, returned to the analytic surfaces of the boundary faces containing the edge, inheriting their side sets and the node sets common to the ends of a boundary edge, relaxed locally with the cavity fixed before the test; on a boundary without an analytic surface it stays at the midpoint.
- *Boundary edge swap*: a boundary edge whose two boundary faces lie in one side set (or in none) and make an angle below `boundary swap angle` is replaced by the edge between the far nodes of the two faces, the faces by the two that contain the new edge, in the side set as well, and the chain of elements behind them by a triangulation of the resulting polygon. *Face swap*: an interior face shared by two elements into the three elements around the edge between their apexes; off by default (Section 11.3).
- *Acceptance test* (`accept_proposal`): the energy of the new elements below that of the old ones by a relative margin (default 10^{-8}), no new element above the allowed density, and the geometric floor: no new element with a scaled Jacobian below `minimum scaled Jacobian` unless the worst element of the cavity was already below it and the new worst is no worse. Under `size criterion: length` the splits and collapses of edges outside the band are accepted on the length alone (Section 7.2); under `shape criterion: scaled Jacobian` the swaps and the shape-driven collapses and splits must raise the cavity minimum of the scaled Jacobian (Section 11.3). In both the energy remains a validity check, and under a metric every shape measure is taken in the metric space.
- *Driver*: candidates above the desired density dilated by adjacency layers; swaps in decreasing order of ring energy with deferral of cavities changed earlier in the pass; with a prescribed target the edges outside the band $[1/\sqrt{2}, \sqrt{2}]$ collapsed or split in every pass (before the swaps with `size operations first`, the splits longest first in the target), and shape-driven collapses and splits only in a pass without accepted swaps unless `shape operations every pass`; no operation may create an edge or a face that exists or was created earlier in the pass; compaction after every operator; refused proposals remembered for the phase and retried only when their cavity changes; passes until none accepts or a cap; the adapted mesh written (as `<output name>-adapted-<k>.g`) and smoothed again until a topology phase accepts nothing or the outer iterations are exhausted, the outputs of the smoothing phases after the first written with the suffixes `-s_0002`, `-s_0003`, and so on, and their times continued from one file to the next, so that ParaView opens them as one sequence. The smoothing phases end on the energy stagnation exit of the solver (`energy stagnation window` and `tolerance`). Input: the top-level `adaptivity` block (`docs/src/reference/model.md`).
- *Tests* 127 to 131, 133, and 134: the kernel on hand-built cavities against an independent evaluation and against the assembled energy; the topology (Euler characteristic, boundary faces against side sets, Exodus round trip with the side numbering); each operator on a hand-built cavity and in topology phases on the distorted cube and on the tube, with the invariants checked (Euler characteristic, closed boundary, side sets covering it, new boundary nodes on the analytic surfaces to 10^{-8}, every node on a face of the cube in the node set of that face, energy decrease accounted for); the length criterion on the tube toward a finer and a coarser target (133); and the options, the ranking, the quality in the metric space, the face swap, the boundary swap, the guard against created entities, and a topology phase on the cube under three combinations of the options (134).

11.2 The distorted cube

Figure 1 records the scaled Jacobian of every element, in the definition of Verdict (the vertex Jacobian times $\sqrt{2}$ over the largest product of the three edge lengths at a vertex), and the energy after every L-BFGS step of the adaptive cube example (`examples/ems/awful-cube`): the distorted cube, 6955 elements, prescribed size 0.214 (the edge of the regular tetrahedron with the mean volume of the input mesh), desired density 0.05, floor 0.35, two adjacency layers, under the energy criterion for every operation and Dirichlet conditions on the node sets of the six faces. This is the first configuration of the loop; the example now runs the configuration of Section 11.3, and the figures of this subsection are kept for the record of what the energy criterion alone achieves.

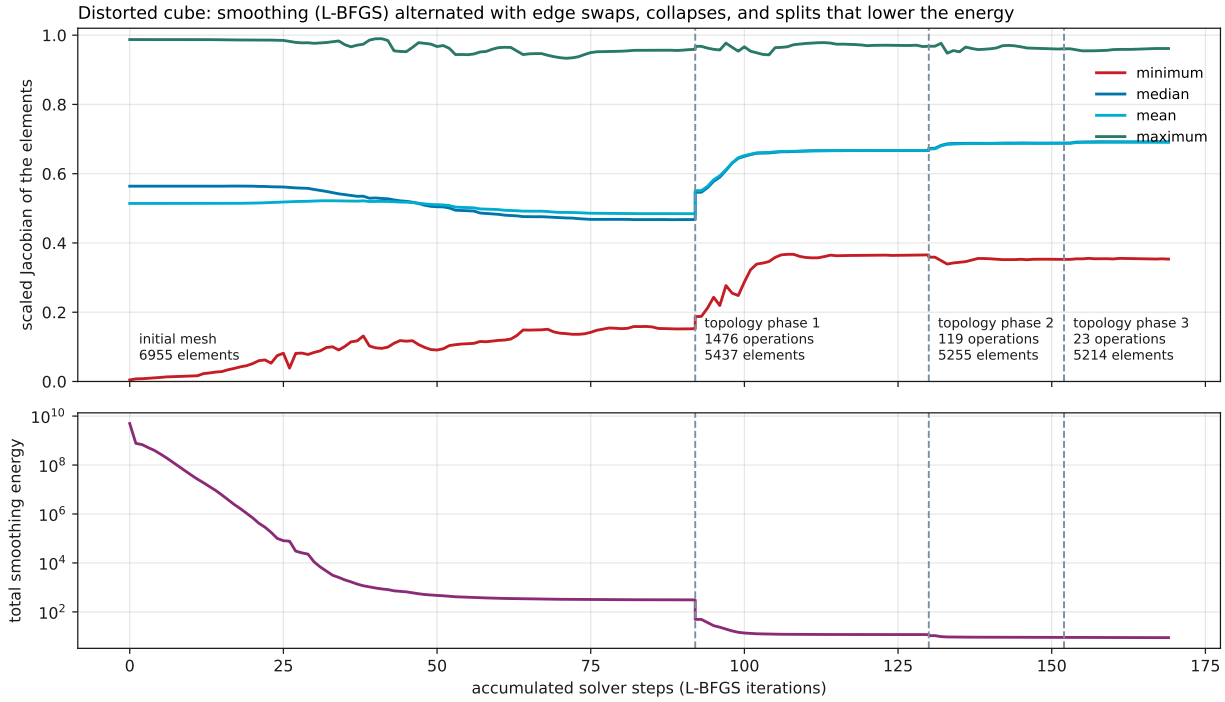


Figure 1: Distorted cube: minimum, median, mean, and maximum scaled Jacobian of the elements (top) and total smoothing energy (bottom) after every solver step; the dashed lines are the topology phases, with the operations accepted and the element count after them.

Smoothing alone raises the minimum scaled Jacobian from 0.004 to 0.15 and reduces the energy from 3.5×10^9 to 3.1×10^2, then stalls at step 92; it lowers the median from 0.56 to 0.47 while it repairs the worst elements, since with a fixed connectivity repairing a sliver deforms its neighbors (Figure 2). The first topology phase accepts 1476 operations at fixed nodes: the minimum rises to 0.19 and the energy falls to 49. The smoothing that follows reaches a minimum of 0.35, a median of 0.69, and an energy of 8.9 within about twenty steps. Two more alternations accept 119 and 23 operations and change little; the process has converged after 169 solver steps, with 5214 elements. Three earlier variants measured on the way: swaps alone reach an energy of 13.2 and a minimum of 0.36; swaps and collapses 8.6 and 0.39; all three operators without the floor 8.5 and 0.29, the splits having created a few flat elements whose smaller size penalty outweighed their shape. The floor is what keeps a single element from being sacrificed to the cavity sum.

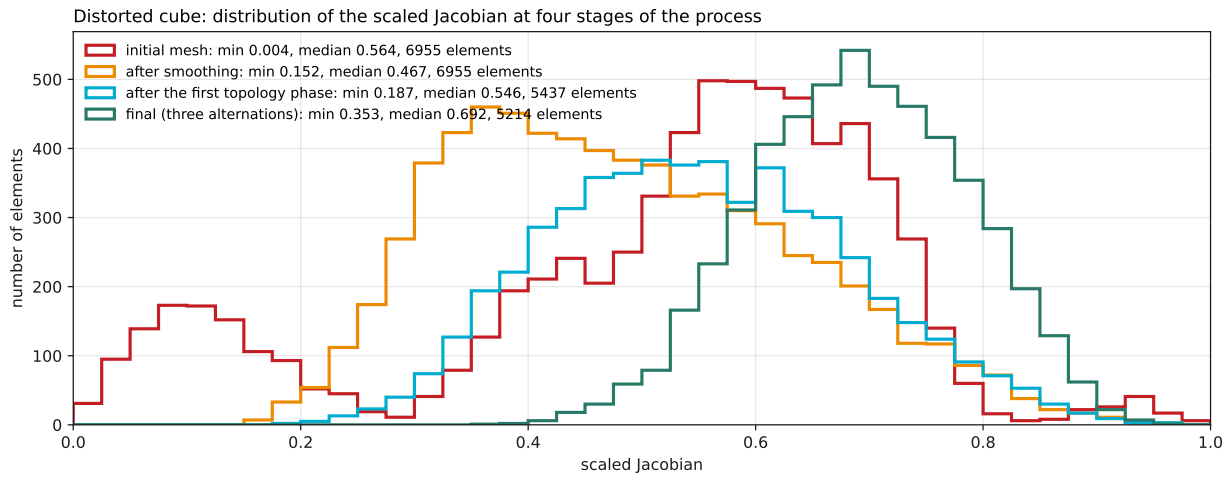


Figure 2: Distorted cube: distribution of the scaled Jacobian in the initial mesh, after smoothing alone, after the first topology phase, and at the end.

11.3 Comparison with the staggered workflow

The workflow the loop replaces alternates Norma smoothing with the Sierra tool `improve_mesh` (Section 4). I ran it on the distorted cube on rigel, five alternations of the same smoothing input as the coupled loop and `improve_mesh` with the target 0.214 and its defaults (band $[1/\sqrt{2}, \sqrt{2}]$, quality metric the scaled Jacobian, up to 20 outer iterations, termination when the minimum stops improving), and the coupled loop with the length criterion for the size operations, a floor of 0.15, and eight adaptivity iterations. Two findings preceded the comparison proper.

Boundary conditions must be given on side sets. `improve_mesh` declares the nodes it creates with no parts (`StkEmendMesh::create_node`), so a new boundary node belongs to no node set, while the sides it creates or swaps carry the side set parts of the sides they replace. With Dirichlet conditions on node sets, Norma does not hold the new nodes: after the first pass 15 to 27 nodes per face of the cube lay on the face outside its node set, and after five alternations 144 boundary nodes were off every face and the cube had bulged to 1.14 in x and 1.17 in y. That run reported a minimum scaled Jacobian of 0.42 and a mean of 0.706, bought by the deformation. With side sets on the six faces and Surface conditions on them, which the loop's own splits, collapses, and boundary swaps also maintain, `improve_mesh` keeps every boundary face in its side set and on its plane. Every number below is from that setting, on the same input mesh (the cube with side sets added), on the same machine; the run-to-run variation from floating point paths is about 0.03 in the minimum and 0.005 in the mean.

The gap was on the boundary. Under the energy criterion the coupled loop ended at 8066 elements, minimum 0.36, mean 0.668, against 8508, 0.38, 0.703 for the staggered workflow. The interior elements were nearly at par (mean 0.681 against 0.709) and the boundary elements were not (0.652 against 0.695), with 72 to 88 percent of the fifty worst elements of the coupled mesh on the boundary. Applying `improve_mesh` once to the converged coupled mesh and smoothing again lifted the minimum from 0.36 to 0.42 and the mean from 0.671 to 0.689, so the difference lay in the topology operators, not in the smoothing, which is Norma's in both workflows. Reading the `emend` sources gave three candidates: it accepts an operation when the cavity minimum of

the scaled Jacobian rises (`is_first_quality_metric_better_than_second_with_tolerance`), not when an energy sum falls; it swaps boundary edges within flat patches, re-triangulating the surface, whereas the loop swapped interior edges only, so that the boundary triangles bisected by the splits were never repaired; and it swaps faces (two elements into the three around the edge between their apexes). The candidate set was not a cause: making every element a candidate changed nothing.

Measured remedies. Each candidate is now an option of the `adaptivity` block, and Table 3 measures them. The scaled Jacobian criterion (`shape criterion: scaled Jacobian`) recovers

	elements	min	mean	interior	boundary
staggered: Norma smoothing and `improve_mesh`	8508	0.38	0.703	0.709	0.695
coupled, energy criterion	8066	0.36	0.668	0.681	0.652
coupled, scaled Jacobian criterion	8356	0.37	0.689	0.704	0.668
coupled, energy criterion, boundary swaps	8048	0.36	0.674	0.683	0.663
coupled, scaled Jacobian criterion, boundary swaps	8258	0.41	0.692	0.701	0.681
the same with face swaps	10146	0.34	0.684	0.691	0.675

Table 3: Distorted cube, target 0.214, Surface conditions on the six faces, Seth–Hill smoothing in Norma for every row: the staggered workflow against the coupled loop with the length criterion for the size operations and a floor of 0.15: the minimum and the mean of the scaled Jacobian over all elements, and its mean over the interior and the boundary elements.

most of the interior; the boundary swaps (`boundary swaps: true`) recover the boundary; together they give a higher minimum than the staggered workflow, the interior at par, and a boundary mean within 0.015, at the edge of the run-to-run variation. Face swaps lower the quality under both criteria, add elements, and keep the loop churning (203 operations at the eighth iteration), so they are off by default. The example `awful-cube-adaptive.yaml` now runs this configuration, and the slides of the study record its evolution: the first topology phase accepts 3055 operations, eight phases in all, 282 solver steps, a minimum of 0.41 and a mean of 0.69 with 8258 elements. Under the scaled Jacobian criterion the energy is no longer the acceptance test of the swaps, but it remains their validity check and the objective of the smoothing; Proposition 1 then covers the smoothing phases alone, and the termination of the topology phases rests on the strict rise of the cavity minimum, which bounds the number of operations as in `emend`.

The residual and the cost. Two more measurements, made on sirius after the runs on rigel. The loop applied to the mesh that the staggered workflow produced improves it further, from a minimum of 0.38 and a mean of 0.703 to 0.44 and 0.707, so the acceptance test admits nothing harmful and the operators are at least as capable; the residual between the two workflows from the distorted input is the path each takes. Of the differences in path, the order of the operators within a pass is the one that matters: `improve_mesh` refines and collapses to the size band before it swaps, and with `size operations first: true` the loop reaches a mean of 0.698 and a minimum of 0.41 from 0.692 and 0.39, against 0.703 and 0.38 for the staggered workflow. Ranking the cavities worst first (which the scaled Jacobian criterion now does), more alternation with fewer passes per phase, and the shape-driven operations in every pass were each measured and gain nothing. On one machine the loop is deterministic, so these differences are real, but between machines the floating point path changes the minimum by about 0.03 and the mean by 0.005, which is the size of what remains.

The cost was a separate matter. On the plate the loop spent ten of its twelve hours at the

last stage in the topology phases and most of the rest in smoothing runs at twelve seconds per L-BFGS iteration on half a million elements. Profiles on rigel found two implementation defects rather than anything in the algorithm. The force evaluation of a smoothing model allocated 19 KB per element and scaled by only a factor of two on sixteen threads: the smoothing reference was built from column slices and dynamic matrices, the size field was called through an untyped function field, and the tangent by automatic differentiation used a runtime-sized vector, which left its return type, and with it the whole constitutive tuple even without a tangent, inferred as `Any`, so that everything downstream was dynamic. With a static reference, a typed barrier for the size field, and a static input to the differentiation, an evaluation of 85 thousand elements goes from 1.48 s and 1.5 GB to 0.11 s and 44 MB, and on 505 thousand elements from 11.1 to 0.9 microseconds per element on sixteen threads. The topology phase tried every candidate edge in every pass and computed the energies of every configuration of a swap; nodes do not move within a phase, so a refused proposal is now remembered for the phase and retried only when an element around it has changed, the configurations of a swap are scored by the scaled Jacobian with the energy of the chosen one computed once as the validity check, and the candidates under that criterion are the elements below `desired scaled Jacobian`. On the converged stage 4 mesh the first swap pass goes from 197 to 92 s and the following passes from about 200 to 7 s; the first three stages of the plate, rerun, took 3, 7, and 39 minutes.

Quality in the metric space, and the order of the splits. Rerunning the anisotropic examples with the faster code reproduced their energies step for step, but the loop under a metric target did not converge: on the tube with the graded axial size of the anisotropic note the operations per iteration stayed at 220 to 440 and the smoothed energy rose from 795 to 3061. The shape measures were the scaled Jacobian in physical space, which penalizes the elongation the target asks for, so the swaps undid what the smoother did. Every shape measure of the loop, the criterion, the floor, the candidates, and the ranking, is now the scaled Jacobian of the element mapped by the metric factor $\boldsymbol{F}_M$ sampled on it, in which an element that matches the target is regular; without a metric the measure is unchanged, since a scalar size scales all edges alike. The tube then converges (167, 72, 49, 37, 20, 13 operations), from 5657 to 6043 elements, with the metric quality from a minimum of 0.17 and a mean of 0.66 to 0.43 and 0.77, 99.6 percent of the edges inside the band in the metric against 87.5 percent on the input, and every boundary node on its surface (`examples/ems/tube/tube-metric-adaptive.yaml`). On the plate the order of the operators gained nothing, refinement being what dominates there, but the order of the splits did: they are tried longest first in the target, as in longest-edge bisection, which is what bounds the shape of the children: the four stages then reach means of 0.682, 0.692, 0.701, and 0.702 with minima of 0.39, 0.38, 0.33, and 0.32, from 0.671, 0.680, 0.693, 0.696 and 0.37, 0.32, 0.22, 0.33 before, against 0.690, 0.700, 0.709, 0.713 and 0.40, 0.37, 0.35, 0.32 for the staggered workflow, with 3389, 15043, 81616, and 486814 elements. The remaining 0.01 in the mean is the subdivision itself: `improve_mesh` refines an element whose edges are all long with a template, one into eight, whereas bisection one edge at a time gives the same count with worse shapes. With the faster code the four stages of the plate take 40 minutes on 16 threads, against about 14 hours before.

Consistency of a pass. Face swaps exposed two hazards in the pass structure that the interior edge swaps had escaped by the shape of their cavities. The operations of one operator are independent, since an accepted operation marks its cavity dead and a proposal touching a dead element is refused, but the adjacency is rebuilt only at the end of a pass, so two operations could create the same edge or face, and a 3-to-2 flip could create a face that already existed elsewhere, giving a

face shared by four elements. Every swap and collapse now refuses to create an edge or a face that exists in the mesh or was created earlier in the pass, and the topology is compacted after every operator, so the collapses and splits see the elements the swaps made.

11.4 The tube

On the tube (`examples/ems/tube/tube-adaptive.yaml`, target 0.08 against a mesh size of 0.12, nodes sliding on the four analytic surfaces, floor 0.3), the size operations dominate: the first topology phase accepts 906 operations, mostly splits of the edges longer than $\sqrt{2} \times 0.08$, and takes the mesh from 5657 to 8896 elements, with the new boundary nodes placed on the cylinders and the caps; the next two phases accept 76 and 18 operations and end at 9012 elements, under the energy criterion. A mesh at the target has about $(0.12/0.08)^3 \times 5657 \approx 19000$ elements, so this is the stall of Section 7.2: the remaining energy, 2.24, is the size mismatch of the edges whose bisection the cavity test refuses. Under the length criterion, one topology phase of six passes without smoothing takes the fraction of edges inside the band from 0.43 (energy criterion, 8896 elements) to 0.78 (24638 elements; test 133), and the example, now run with `size criterion: length`, a floor of 0.15, and six adaptivity iterations, accepts 8801, 1864, 721, 356, 225, and 134 operations and ends at 25544 elements with 88 percent of the edges inside the band and the median edge ratio 0.97; the scaled Jacobian ends at a minimum of 0.24 and a median of 0.59, against 0.41 and 0.82 for the input mesh, whose edges were 1.5 times the target. The smoothing energy rises from 3.01 to 5.42 over the refinement: with equal moduli the functional charges the shapes of the bisected elements more than it credits their sizes, which is the measured reason the size operations are not judged by it. The test with a target of 0.24 shows the opposite: the short edges are collapsed, the count falls, and the boundary stays closed and covered by the side sets.

11.5 The plate refined toward a sinusoid

The example of `examples/ems/plate` reproduces a thought experiment of the September 2026 program review: a plate of $1 \times 1 \times 0.1$ meshed at the far-field size $h_c = 0.1$, one element through the thickness (689 tetrahedra), and a size field

$$h(x, y) = h_f \left(\frac{h_c}{h_f} \right)^s, \qquad s = 2\,|x - (0.1 \sin 4\pi y + 0.5)|, \tag{6}$$

that falls geometrically from the far field to the near-field size h_f on a sinusoid through the plate, as a localizing deformation would demand. The near-field size is halved in four stages, from 0.05 to 0.00625, each stage running the adaptivity loop on the smoothed mesh of the stage before (`refine.jl`); the size field is given as an expression in x and y and the stage sets h_f in it. The formula makes the size exceed h_c in the corners, where s reaches 1.2; it is used as written. A mesh exactly at the target has about 2900, 12 800, 69 000, and 420 000 elements at the four stages.

The measurements below record how the example came to its present configuration; its current results, with the scaled Jacobian criterion, boundary swaps, the size operations first, and the splits longest first, are at the end of Section 11.3, with the comparison against the staggered workflow and the present cost of 40 minutes for the four stages.

Under the energy criterion, with equal moduli and a floor of 0.35, the first stage stalled at 1068 elements with 920 of its 1680 edges beyond the band, every split raising the cavity energy by 52 percent at the median, and the mesh looked like the input with a few more elements. Under the length criterion, with a floor of 0.15, equal moduli, and the energy criterion for the swaps, the stages gave

stage	h_f	elements	min SJ	median SJ	edges in band
1	0.05	3404	0.24	0.59	89 %
2	0.025	15199	0.21	0.62	91 %
3	0.0125	81721	0.17	0.64	93 %
4	0.00625	487213	0.14	0.65	94 %

with the edge ratio to the target at the median 1.00 and its 5 and 95 percent quantiles at 0.72 and 1.4 throughout; SJ is the scaled Jacobian. Figure 3 shows the top face of the mesh after each stage against the prescribed size. The operations accepted per adaptivity iteration fall by a factor of two to four each time (stage 4: 177294, 41713, 17372, 9470, 5565, 3441, 2342, 1792), and the smoothing energy after each phase is above the energy before it, as on the tube, since the bisected elements cost more in shape than they return in size under equal moduli. The four stages took 4, 7, 43, and 230 minutes on 16 threads before the performance work of Section 11.3; the last is dominated by the eight smoothing phases of up to 200 L-BFGS iterations on half a million elements. The element count exceeds that of a mesh at the target by 15 to 20 percent at every stage, the overshoot of bisection: an edge just beyond $\sqrt{2}$ is split into halves just above $1/\sqrt{2}$, below the target.

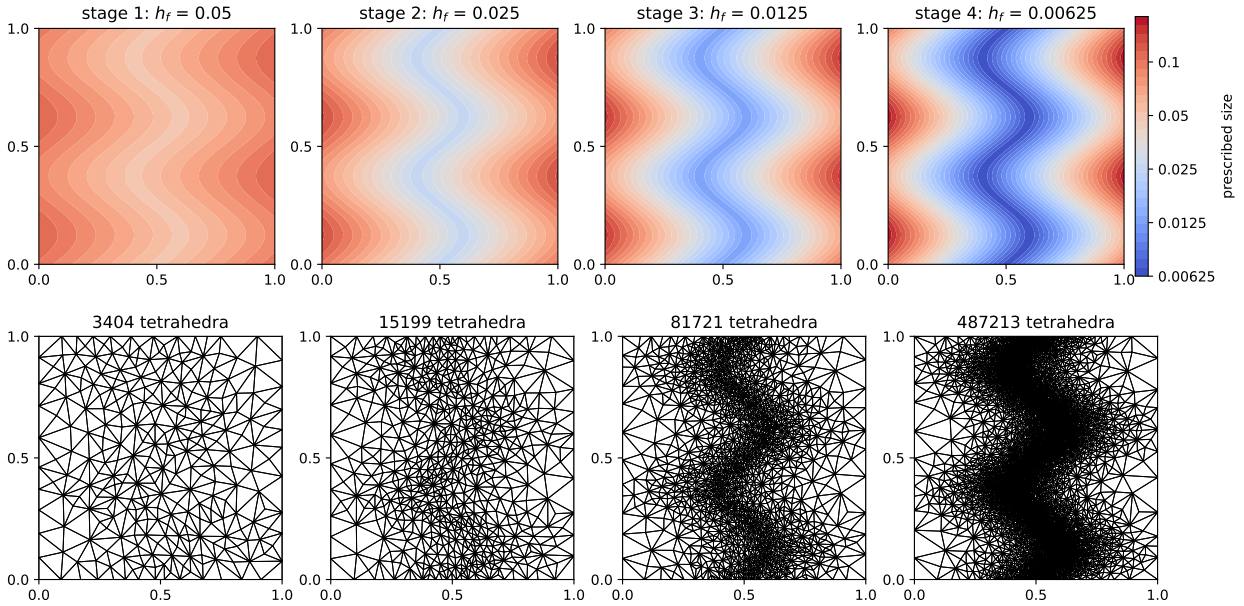


Figure 3: Plate refined toward a sinusoid: the prescribed size field and the top face of the mesh after each of the four stages.

12 Risks and open questions

The risks this note ranked before the implementation, and how each was resolved, followed by the questions that remain open.

1. *The acceptance test and the kernel.* The kernel was validated on hand-built cavities, against an independent evaluation and the assembled energy, before the loop was built (test 127). The single functional then proved the wrong test for two classes of operation: the size operations toward a target, refused by the cavity sum beyond a ratio of about 1.5 (Section 7.2), and

the swaps, better judged by the cavity minimum of the scaled Jacobian (Section 11.3). The energy remains the objective of the smoothing and the validity check of every operation.

2. *Boundary collapses on curved surfaces.* Resolved by construction: a collapse keeps the position of the surviving node, which must lie on every surface of the removed node, so no projection follows the test.
3. *Four-node tetrahedra only.* A decision, stated in the input documentation; higher-order meshes are produced by conversion after the loop.
4. *Energy thresholds across target sizes.* The thresholds are on the density and the cavity comparison on the sums (Section 7), as planned.
5. *Geometric floor versus energy test.* The floor is part of the acceptance test under every criterion.

Open questions:

1. *Subdivision.* On the plate the loop remains about 0.01 below the staggered workflow in the mean scaled Jacobian. The difference is the subdivision: `improve_mesh` refines an element whose edges are all long with a template, one into eight, whereas bisection one edge at a time gives the same count with worse shapes.
2. *Termination under the scaled Jacobian criterion.* Descent of the energy (Proposition 1) then covers the smoothing phases alone; the topology phases terminate on the strict rise of the cavity minimum, which bounds the number of operations but not the energy across a phase.
3. *Over-constrained targets.* A target that asks for more elements across a direction bounded by fixed faces than the mesh has is a strict local minimum of the smoothing; the topology reaches it (`docs/notes/ems-anisotropic`), but the loop does not detect the situation, and a smoothing run alone reports convergence.
4. *A metric from a solution.* The loop takes its target as given; the construction of a metric from the recovered Hessian of a field is the next step of the anisotropic note.

13 Summary

The coupled loop rests on one design decision and three algorithms. The decision is to use a single distortion functional for both phases: the metric-conditioned smoothing energy (1) when a size or metric field is prescribed, which is the common case for a preprocessing step and makes size-driven operations decreases of the same objective, and the intrinsic equal-volume functional (2) otherwise, so that in either case one quantity is meaningful across every smoothing step and every topological operation. The loop is a preprocessing step on four-node tetrahedra; it transfers no solution state, and higher-order meshes are obtained by conversion afterwards. Of the alternatives, wrapping Omega_h would reproduce the present staggered workflow with a new dependency, and the Sierra hand-off cannot be part of a public workflow; both remain useful as references for validation. The algorithms are: the transplanted Omega_h trigger logic—two-tier thresholds on the energy density, candidate dilation by adjacency layers, swaps before collapses, progress-based termination—; the acceptance test (4), which strengthens the strict-improvement filter of Omega_h from the cavity minimum to the cavity sum and thereby makes local acceptance imply global descent (Proposition 1), with two measured exceptions: the size operations toward a prescribed target, which the cavity test refuses beyond a ratio of about 1.5 because a bisection halves the

shape quality before the smoothing repairs it, are accepted on the edge length in the target, within a band that no collapse or split may leave (Section 7.2); and the swaps, which the comparison with the staggered workflow of Norma smoothing and `improve_mesh` (Section 11.3) showed are better accepted on the cavity minimum of the scaled Jacobian, and which must include the boundary edges of flat patches, since the loop was behind that workflow on the boundary alone and matches it with both changes; and the windowed stagnation test (5) with explicit detection of the topological lock, which supplies both the trigger for the topology phase and the terminator of the outer loop. Norma already provided the per-element energy field, the metric-space ideal element, the analytic surface constraints, and the mesh-replacement orchestration; the new construction is the in-memory topology cache with adjacency, the three cavity operators, and the single acceptance test that all of them pass. All of it is in place as of September 22, 2026, with every shape measure taken in the metric space when the target is a metric; the measurements of Section 11 confirm the premise of the note, that the alternation improves the mesh far beyond either mechanism alone.

References

[1] D. A. Ibanez et al. Omega_h: reliable mesh adaptation. Sandia National Laboratories, `https://github.com/sandialabs/omega_h`.

[2] D. A. Ibanez. *Conformal Mesh Adaptation on Heterogeneous Supercomputers*. PhD thesis, Rensselaer Polytechnic Institute, 2016.

[3] L. A. Freitag, C. Ollivier-Gooch. Tetrahedral mesh improvement using swapping and smoothing. *International Journal for Numerical Methods in Engineering* 40 (1997) 3979–4002.

[4] G. T. Klincsek. Minimal triangulations of polygonal domains. *Annals of Discrete Mathematics* 9 (1980) 121–123.

[5] B. M. Klingner, J. R. Shewchuk. Aggressive tetrahedral mesh improvement. In: *Proceedings of the 16th International Meshing Roundtable*, Springer, 2007, 3–23.

[6] J. Nocedal, S. J. Wright. *Numerical Optimization*, 2nd edition. Springer, 2006.

[7] A. Mota, W. Sun, J. T. Ostien, J. W. Foulk III, K. N. Long. Lie-group interpolation and variational recovery for internal variables. *Computational Mechanics* 52 (2013) 1281–1299.